

Appendix A Installing Python

Downloading Python

To run the programs shown in this book, you will need to install Python 3.0, or a later version. You can download the latest version of Python from www.python.org/downloads. This appendix discusses installing Python for Windows. Python is also available for the Mac, Linux, and several other platforms. Links to download versions of Python for these systems are shown on the Python download site at www.python.org/downloads.



Keep in mind that there are two *families* of Python that you can download: Python 3.x and Python 2.x. The programs in this book work only with the Python 3.x family.

Installing Python 3.x For Windows

When you visit the Python download site at www.python.org/downloads, you should download the latest version of Python 3.x that is available. **Figure A-1**  shows how the download site appeared at the time this was written. As you can see in the figure, Python 3.5.2 was the latest version at that time.

Figure A-1 Download the latest Python version



Once you have downloaded the Python installer, you should execute it. **Figure A-2** shows the installer for Python 3.5.2. It is highly recommended that you check both options at the bottom of the screen: *Install launcher for all users*, and *Add Python 3.x to PATH*. Once you have done that, click *Install Now*.

Figure A-2 Python installer



Next, Windows will prompt you with a message such as “Do you want to allow this app to make changes to your device?” Click Yes to proceed with the installation. When the installation has finished, you will see the message “Installation was successful.” Click the Close button to exit the installer.

Appendix B Introduction to IDLE

IDLE is an integrated development environment that combines several development tools into one program, including the following:



Introduction to IDLE

- A Python shell running in interactive mode. You can type Python statements at the shell prompt and immediately execute them. You can also run complete Python programs.
- A text editor that color codes Python keywords and other parts of programs.
- A “check module” tool that checks a Python program for syntax errors without running the program.
- Search tools that allow you to find text in one or more files.
- Text formatting tools that help you maintain consistent indentation levels in a Python program.
- A debugger that allows you to single-step through a Python program and watch the values of variables change as each

statement executes.

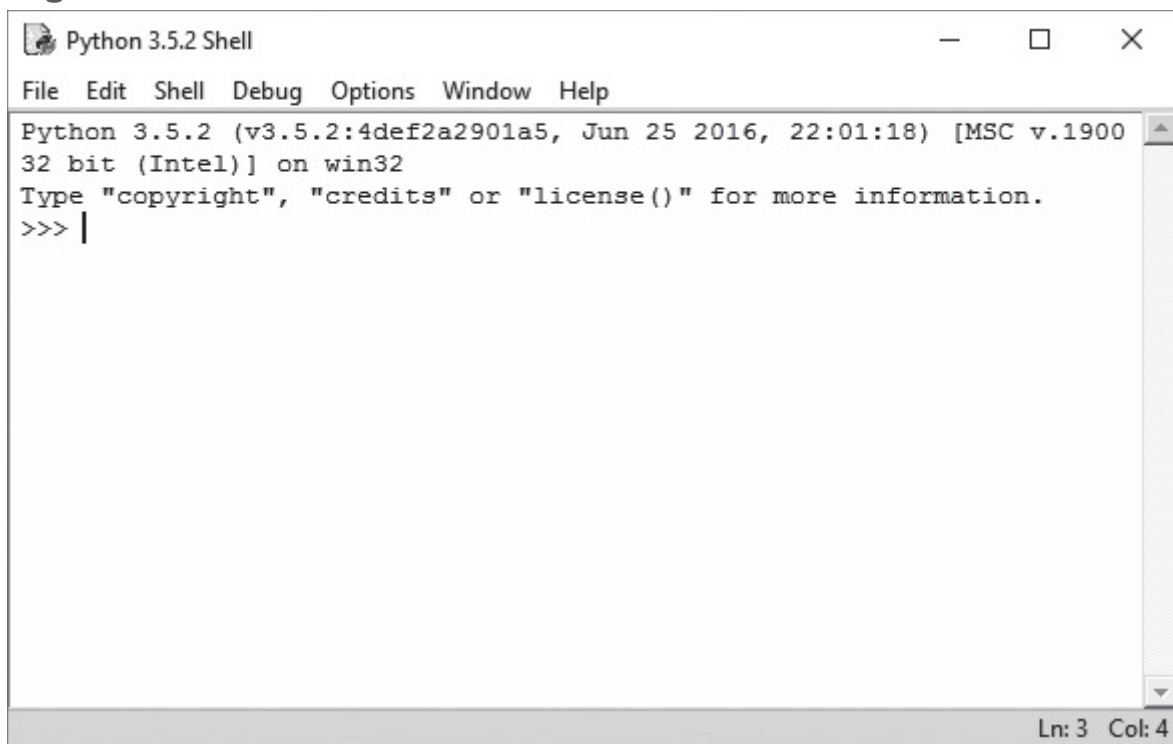
- Several other advanced tools for developers.

The IDLE software is bundled with Python. When you install the Python interpreter, IDLE is automatically installed as well. This appendix provides a quick introduction to IDLE, and describes the basic steps of creating, saving, and executing a Python program.

Starting IDLE and Using the Python Shell

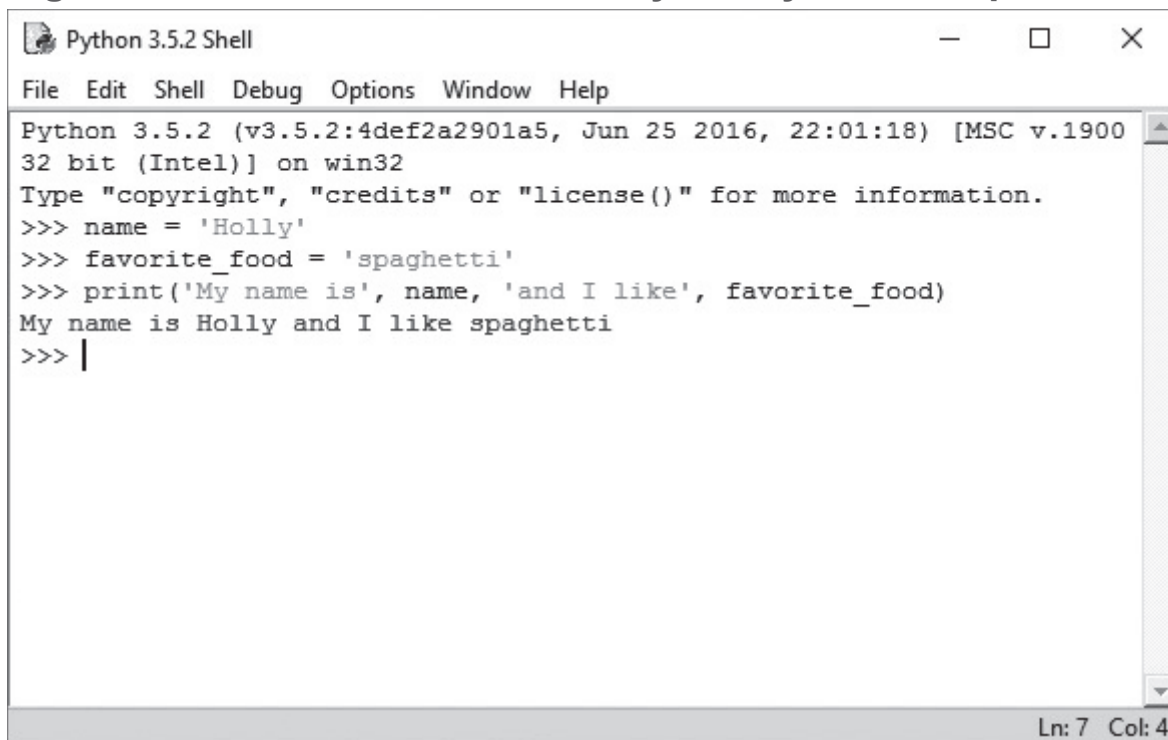
After Python is installed on your system, a Python program group will appear in your Start menu's program list. One of the items in the program group will be titled *IDLE (Python GUI)*. Click this item to start IDLE, and you will see the Python Shell window shown in [Figure B-1](#). Inside this window, the Python interpreter is running in interactive mode, and at the top of the window is a menu bar that provides access to all of IDLE's tools.

Figure B-1 IDLE shell window



The `>>>` prompt indicates that the interpreter is waiting for you to type a Python statement. When you type a statement at the `>>>` prompt and press the Enter key, the statement is immediately executed. For example, [Figure B-2](#) shows the Python Shell window after three statements have been entered and executed.

Figure B-2 Statements executed by the Python interpreter

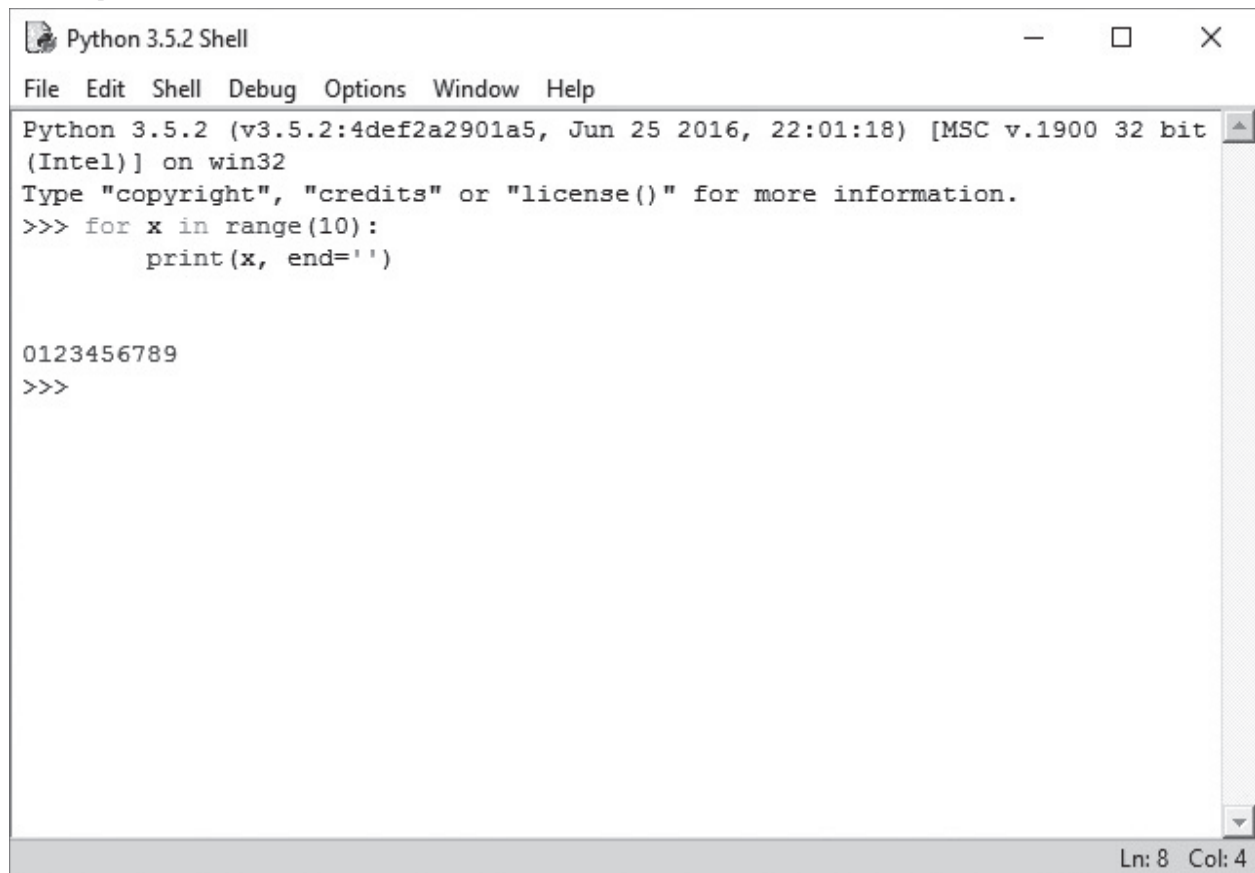


```
Python 3.5.2 Shell
File Edit Shell Debug Options Window Help
Python 3.5.2 (v3.5.2:4def2a2901a5, Jun 25 2016, 22:01:18) [MSC v.1900
32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> name = 'Holly'
>>> favorite_food = 'spaghetti'
>>> print('My name is', name, 'and I like', favorite_food)
My name is Holly and I like spaghetti
>>> |
```

When you type the beginning of a multiline statement, such as an `if` statement or a loop, each subsequent line is automatically indented. Pressing the Enter key on an empty line indicates the end of the multiline statement and causes the interpreter to execute it. [Figure B-3](#) shows the Python Shell window after a `for` loop has been entered and executed.

Figure B-3 A multiline statement executed by the Python

interpreter

A screenshot of a Python 3.5.2 Shell window. The window has a title bar with the text "Python 3.5.2 Shell" and standard minimize, maximize, and close buttons. Below the title bar is a menu bar with the following items: File, Edit, Shell, Debug, Options, Window, and Help. The main area of the window contains the following text:

```
Python 3.5.2 (v3.5.2:4def2a2901a5, Jun 25 2016, 22:01:18) [MSC v.1900 32 bit  
(Intel)] on win32  
Type "copyright", "credits" or "license()" for more information.  
>>> for x in range(10):  
    print(x, end=' ')
```

The output of the code is displayed on the next line:

```
0123456789  
>>>
```

At the bottom right of the window, the status bar shows "Ln: 8 Col: 4".

Writing a Python Program in the IDLE Editor

To write a new Python program in IDLE, you open a new editing window. As shown in [Figure B-4](#), you click File on the menu bar, then click New File on the dropdown menu. (Alternatively, you can press Ctrl+N.) This opens a text editing window like the one shown in [Figure B-5](#).

Figure B-4 The File menu

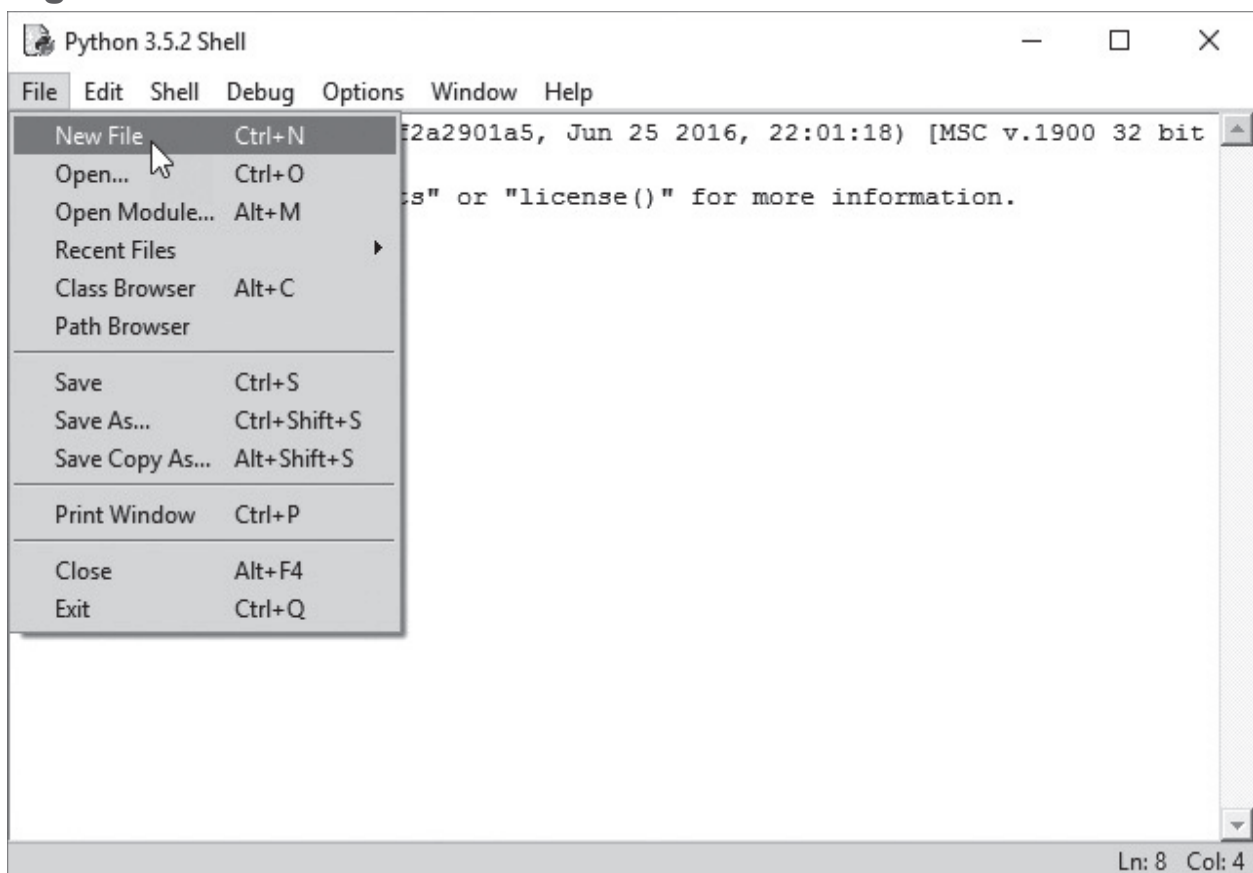
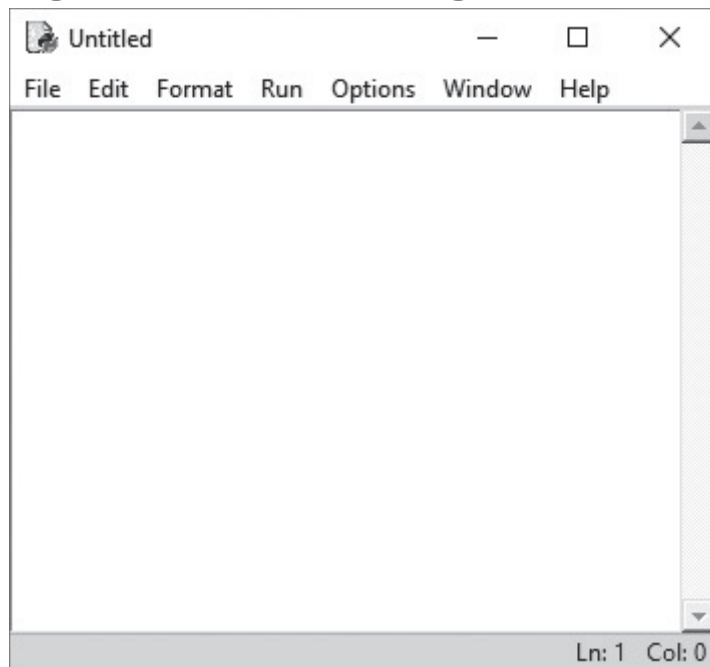


Figure B-5 A text editing window



To open a program that already exists, click File on the menu bar, then click Open. Simply browse to the file's location and select it, and it will be opened in an editor window.

Color Coding

Code that is typed into the editor window, as well as in the Python Shell window, is colorized as follows:

- Python keywords are displayed in orange.
- Comments are displayed in red.
- String literals are displayed in green.
- Defined names, such as the names of functions and classes, are displayed in blue.
- Built-in functions are displayed in purple.

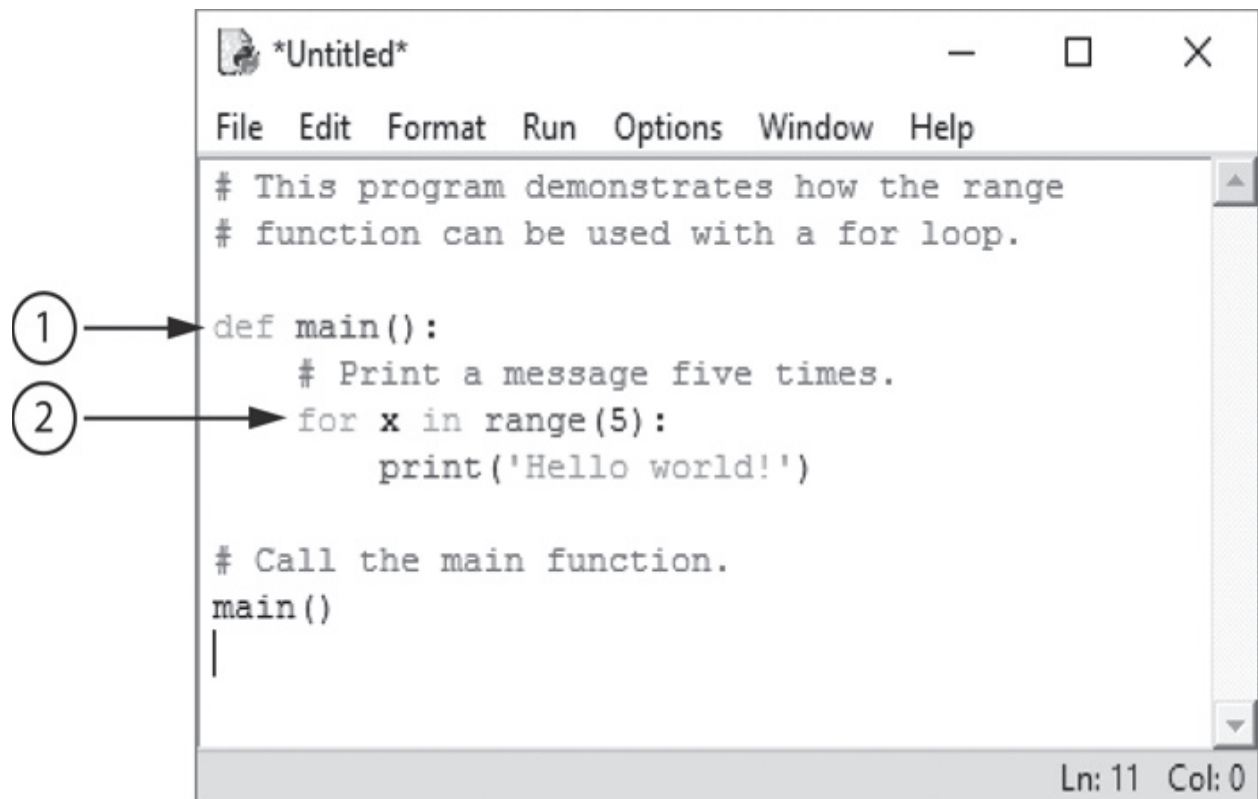


You can change IDLE's color settings by clicking Options on the menu bar, then clicking Configure IDLE. Select the Highlighting tab at the top of the dialog box, and you can specify colors for each element of a Python program.

Automatic Indentation

The IDLE editor has features that help you to maintain consistent indentation in your Python programs. Perhaps the most helpful of these features is automatic indentation. When you type a line that ends with a colon, such as an `if` clause, the first line of a loop, or a function header, then press the Enter key, the editor automatically indents the lines that are entered next. For example, suppose you are typing the code shown in [Figure B-6](#) . After you press the Enter key at the end of the line marked ①, the editor will automatically indent the lines that you type next. Then, after you press the Enter key at the end of the line marked ②, the editor indents again. Pressing the Backspace key at the beginning of an indented line cancels one level of indentation.

Figure B-6 Lines that cause automatic indentation



The screenshot shows the IDLE Python IDE window titled '*Untitled*'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
# This program demonstrates how the range
# function can be used with a for loop.

def main():
    # Print a message five times.
    for x in range(5):
        print('Hello world!')

# Call the main function.
main()
|
```

Annotations are present on the left side of the code editor:

- Annotation 1: A circle containing the number 1 with an arrow pointing to the `def main():` line.
- Annotation 2: A circle containing the number 2 with an arrow pointing to the `for x in range(5):` line.

The status bar at the bottom right indicates 'Ln: 11 Col: 0'.

By default, IDLE indents four spaces for each level of indentation. It is possible to change the number of spaces by clicking Options on the menu bar, then clicking Configure IDLE. Make sure Fonts/Tabs is selected at the top of the dialog box, and you will see a slider bar that allows you to change the number of spaces used for indentation width. However, because four spaces is the standard width for indentation in Python, it is recommended that you keep this setting.

Saving a Program

In the editor window, you can save the current program by selecting any of these operations from the File menu:

- Save
- Save As
- Save Copy As

The Save and Save As operations work just as they do in any Windows application. The Save Copy As operation works like Save As, but it leaves the original program in the editor window.

Running a Program

Once you have typed a program into the editor, you can run it by pressing the F5 key, or as shown in [Figure B-7](#), by clicking Run on the editor window's menu bar, then clicking Run Module. If the program has not been saved since the last modification was made, you will see the dialog box shown in [Figure B-8](#). Click OK to save the program. When the program runs, you will see its output displayed in IDLE's Python Shell window, as shown in [Figure B-9](#).

Figure B-7 The editor window's Run menu

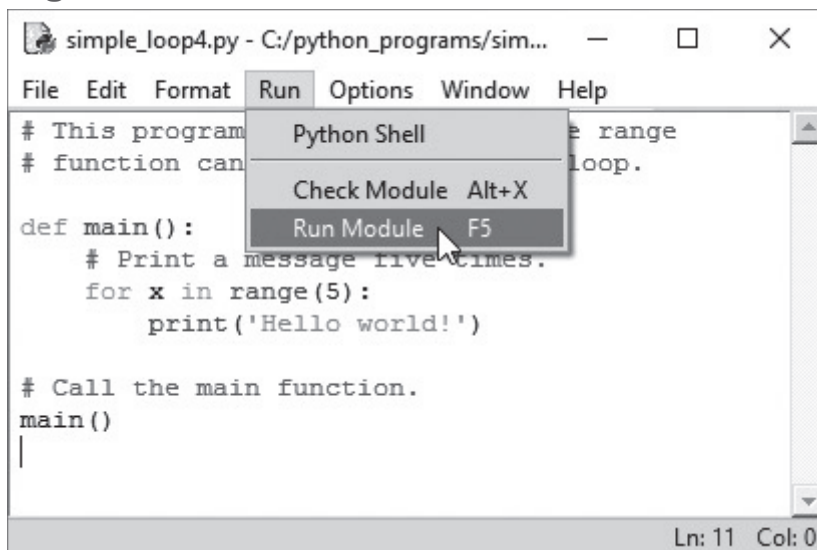


Figure B-8 Save confirmation dialog box

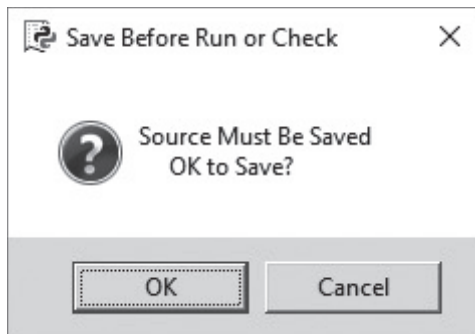
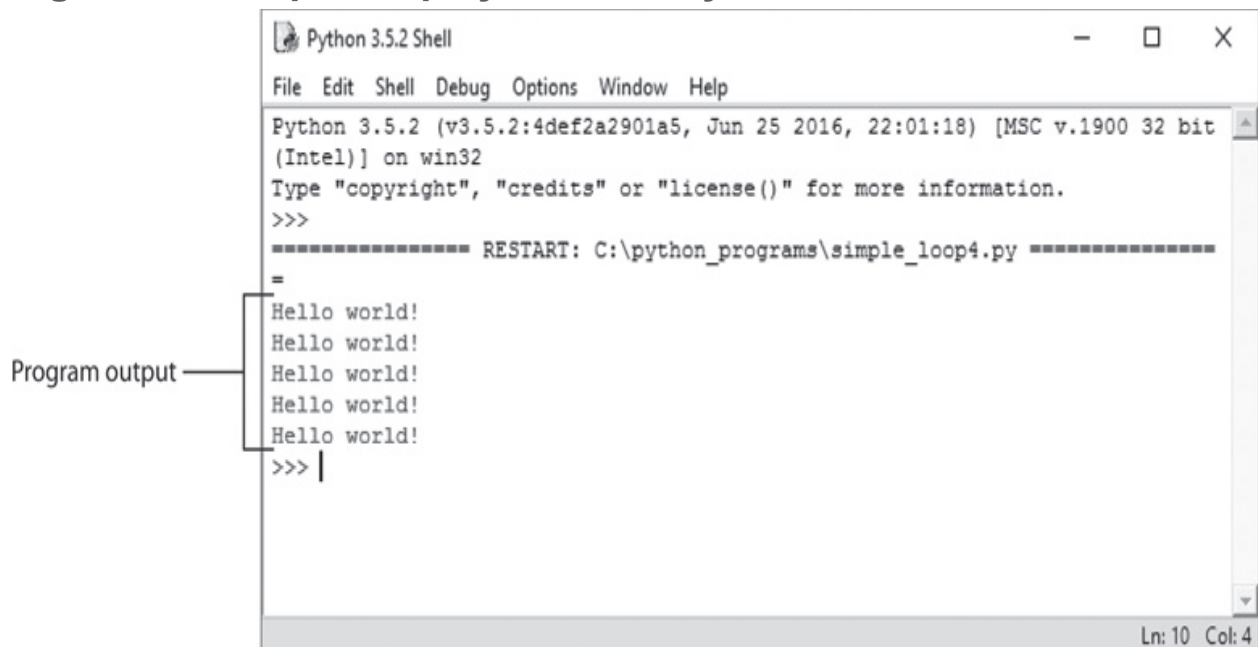
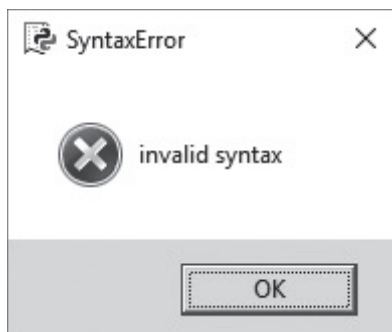


Figure B-9 Output displayed in the Python Shell window



If a program contains a syntax error, when you run the program you will see the dialog box shown in [Figure B-10](#). After you click the OK button, the editor will highlight the location of the error in the code. If you want to check the syntax of a program without trying to run it, you can click Run on the menu bar, then click Check Module. Any syntax errors that are found will be reported.

Figure B-10 Dialog box reporting a syntax error



Other Resources

This appendix has provided an overview for using IDLE to create, save, and execute programs. IDLE provides many more advanced features. To read about additional capabilities, see the official IDLE documentation at www.python.org/idle

Appendix C The ASCII Character Set

The following table lists the ASCII (American Standard Code for Information Interchange) character set, which is the same as the first 127 Unicode character codes. This group of character codes is known as the *Latin Subset of Unicode*. The code columns show character codes, and the character columns show the corresponding characters. For example, the code 65 represents the letter A. Note that the first 31 codes, and code 127, represent control characters that are not printable.

Code	Character	Code	Character	Code	Character	Code	Character	Code	Character
0	NUL	26	SUB	52	4	78	N	104	h
1	SOH	27	Escape	53	5	79	O	105	i
2	STX	28	FS	54	6	80	P	106	j
3	ETX	29	GS	55	7	81	Q	107	k
4	EOT	30	RS	56	8	82	R	108	l
5	ENQ	31	US	57	9	83	S	109	m
6	ACK	32	(Space)	58	:	84	T	110	n
7	BEL	33	!	59	;	85	U	111	o
8	Backspace	34	"	60	<	86	V	112	p
9	HTab	35	#	61	=	87	W	113	q

9	HTab	35	#	61	=	87	W	113	q
10	Line Feed	36	\$	62	>	88	X	114	r
11	VTab	37	%	63	?	89	Y	115	s
12	Form Feed	38	&	64	@	90	Z	116	t
13	CR	39	'	65	A	91	[	117	u
14	SO	40	(	66	B	92	\	118	v
15	SI	41	)	67	C	93	]	119	w
16	DLE	42	*	68	D	94	^	120	x
17	DC1	43	+	69	E	95	—	121	y
18	DC2	44	'	70	F	96	`	122	z
19	DC3	45	—	71	G	97	a	123	{
20	DC4	46	.	72	H	98	b	124	
21	NAK	47	/	73	I	99	c	125	}
22	SYN	48	0	74	J	100	d	126	~
23	ETB	49	1	75	K	101	e	127	DEL
24	CAN	50	2	76	L	102	f		
25	EM	51	3	77	M	103	g		

Appendix D Predefined Named Colors

These are the defined color names that can be used with the turtle graphics library, `matplotlib`, and `tkinter`.

'snow'	'ghost white'	'white smoke'
'gainsboro'	'floral white'	'old lace'
'linen'	'antique white'	'papaya whip'
'blanched almond'	'bisque'	'peach puff'
'navajo white'	'lemon chiffon'	'mint cream'
'azure'	'alice blue'	'lavender'
'lavender blush'	'misty rose'	'dark slate gray'
'dim gray'	'slate gray'	'light slate gray'
'gray'	'light grey'	'midnight blue'
'navy'	'cornflower blue'	'dark slate blue'
'slate blue'	'medium slate blue'	'light slate blue'

'medium blue'	'royal blue'	'blue'
'dodger blue'	'deep sky blue'	'sky blue'
'light sky blue'	'steel blue'	'light steel blue'
'light blue'	'powder blue'	'pale turquoise'
'dark turquoise'	'medium turquoise'	'turquoise'
'cyan'	'light cyan'	'cadet blue'
'medium aquamarine'	'aquamarine'	'dark green'
'dark olive green'	'dark sea green'	'sea green'
'medium sea green'	'light sea green'	'pale green'
'spring green'	'lawn green'	'medium spring green'
'green yellow'	'lime green'	'yellow green'
'forest green'	'olive drab'	'dark khaki'
'khaki'	'pale goldenrod'	'light goldenrod yellow'
'light yellow'	'yellow'	'gold'
'light goldenrod'	'goldenrod'	'dark goldenrod'
'rosy brown'	'indian red'	'saddle brown'
'sandy brown'	'dark salmon'	'salmon'

'light salmon'	'orange'	'dark orange'
'coral'	'light coral'	'tomato'
'orange red'	'red'	'hot pink'
'deep pink'	'pink'	'light pink'
'pale violet red'	'maroon'	'medium violet red'
'violet red'	'medium orchid'	'dark orchid'
'dark violet'	'blue violet'	'purple'
'medium purple'	'thistle'	'snow2'
'snow3'	'snow4'	'seashell2'
'seashell3'	'seashell4'	'AntiqueWhite1'
'AntiqueWhite2'	'AntiqueWhite3'	'AntiqueWhite4'
'bisque2'	'bisque3'	'bisque4'
'PeachPuff2'	'PeachPuff3'	'PeachPuff4'
'NavajoWhite2'	'NavajoWhite3'	'NavajoWhite4'
'LemonChiffon2'	'LemonChiffon3'	'LemonChiffon4'
'cornsilk2'	'cornsilk3'	'cornsilk4'
'ivory2'	'ivory3'	'ivory4'
'honeydew2'	'honeydew3'	'honeydew4'

'LavenderBlush2'	'LavenderBlush3'	'LavenderBlush4'
'MistyRose2'	'MistyRose3'	'MistyRose4'
'azure2'	'azure3'	'azure4'
'SlateBlue1'	'SlateBlue2'	'SlateBlue3'
'SlateBlue4'	'RoyalBlue1'	'RoyalBlue2'
'RoyalBlue3'	'RoyalBlue4'	'blue2'
'blue4'	'DodgerBlue2'	'DodgerBlue3'
'DodgerBlue4'	'SteelBlue1'	'SteelBlue2'
'SteelBlue3'	'SteelBlue4'	'DeepSkyBlue2'
'DeepSkyBlue3'	'DeepSkyBlue4'	'SkyBlue1'
'SkyBlue2'	'SkyBlue3'	'SkyBlue4'
'LightSkyBlue1'	'LightSkyBlue2'	'LightSkyBlue3'
'LightSkyBlue4'	'SlateGray1'	'SlateGray2'
'SlateGray3'	'SlateGray4'	'LightSteelBlue1'
'LightSteelBlue2'	'LightSteelBlue3'	'LightSteelBlue4'
'LightBlue1'	'LightBlue2'	'LightBlue3'
'LightBlue4'	'LightCyan2'	'LightCyan3'
'LightCyan4'	'PaleTurquoise1'	'PaleTurquoise2'

'PaleTurquoise3'	'PaleTurquoise4'	'CadetBlue1'
'CadetBlue2'	'CadetBlue3'	'CadetBlue4'
'turquoise1'	'turquoise2'	'turquoise3'
'turquoise4'	'cyan2'	'cyan3'
'cyan4'	'DarkSlateGray1'	'DarkSlateGray2'
'DarkSlateGray3'	'DarkSlateGray4'	'aquamarine2'
'aquamarine4'	'DarkSeaGreen1'	'DarkSeaGreen2'
'DarkSeaGreen3'	'DarkSeaGreen4'	'SeaGreen1'
'SeaGreen2'	'SeaGreen3'	'PaleGreen1'
'PaleGreen2'	'PaleGreen3'	'PaleGreen4'
'SpringGreen2'	'SpringGreen3'	'SpringGreen4'
'green2'	'green3'	'green4'
'chartreuse2'	'chartreuse3'	'chartreuse4'
'OliveDrab1'	'OliveDrab2'	'OliveDrab4'
'DarkOliveGreen1'	'DarkOliveGreen2'	'DarkOliveGreen3'
'DarkOliveGreen4'	'khaki1'	'khaki2'
'khaki3'	'khaki4'	'LightGoldenrod1'
'LightGoldenrod2'	'LightGoldenrod3'	'LightGoldenrod4'

'LightYellow2'	'LightYellow3'	'LightYellow4'
'yellow2'	'yellow3'	'yellow4'
'gold2'	'gold3'	'gold4'
'goldenrod1'	'goldenrod2'	'goldenrod3'
'goldenrod4'	'DarkGoldenrod1'	'DarkGoldenrod2'
'DarkGoldenrod3'	'DarkGoldenrod4'	'RosyBrown1'
'RosyBrown2'	'RosyBrown3'	'RosyBrown4'
'IndianRed1'	'IndianRed2'	'IndianRed3'
'IndianRed4'	'sienna1'	'sienna2'
'sienna3'	'sienna4'	'burlywood1'
'burlywood2'	'burlywood3'	'burlywood4'
'wheat1'	'wheat2'	'wheat3'
'wheat4'	'tan1'	'tan2'
'tan4'	'chocolate1'	'chocolate2'
'chocolate3'	'firebrick1'	'firebrick2'
'firebrick3'	'firebrick4'	'brown1'
'brown2'	'brown3'	'brown4'
'salmon1'	'salmon2'	'salmon3'

'salmon4'	'LightSalmon2'	'LightSalmon3'
'LightSalmon4'	'orange2'	'orange3'
'orange4'	'DarkOrange1'	'DarkOrange2'
'DarkOrange3'	'DarkOrange4'	'coral1'
'coral2'	'coral3'	'coral4'
'tomato2'	'tomato3'	'tomato4'
'OrangeRed2'	'OrangeRed3'	'OrangeRed4'
'red2'	'red3'	'red4'
'DeepPink2'	'DeepPink3'	'DeepPink4'
'HotPink1'	'HotPink2'	'HotPink3'
'HotPink4'	'pink1'	'pink2'
'pink3'	'pink4'	'LightPink1'
'LightPink2'	'LightPink3'	'LightPink4'
'PaleVioletRed1'	'PaleVioletRed2'	'PaleVioletRed3'
'PaleVioletRed4'	'maroon1'	'maroon2'
'maroon3'	'maroon4'	'VioletRed1'
'VioletRed2'	'VioletRed3'	'VioletRed4'
'magenta2'	'magenta3'	'magenta4'

'orchid1'	'orchid2'	'orchid3'
'orchid4'	'plum1'	'plum2'
'plum3'	'plum4'	'MediumOrchid1'
'MediumOrchid2'	'MediumOrchid3'	'MediumOrchid4'
'DarkOrchid1'	'DarkOrchid2'	'DarkOrchid3'
'DarkOrchid4'	'purple1'	'purple2'
'purple3'	'purple4'	'MediumPurple1'
'MediumPurple2'	'MediumPurple3'	'MediumPurple4'
'thistle1'	'thistle2'	'thistle3'
'thistle4'	'gray1'	'gray2'
'gray3'	'gray4'	'gray5'
'gray6'	'gray7'	'gray8'
'gray9'	'gray10'	'gray11'
'gray12'	'gray13'	'gray14'
'gray15'	'gray16'	'gray17'
'gray18'	'gray19'	'gray20'
'gray21'	'gray22'	'gray23'
'gray24'	'gray25'	'gray26'

'gray27 '	'gray28 '	'gray29 '
'gray30 '	'gray31 '	'gray32 '
'gray33 '	'gray34 '	'gray35 '
'gray36 '	'gray37 '	'gray38 '
'gray39 '	'gray40 '	'gray42 '
'gray43 '	'gray44 '	'gray45 '
'gray46 '	'gray47 '	'gray48 '
'gray49 '	'gray50 '	'gray51 '
'gray52 '	'gray53 '	'gray54 '
'gray55 '	'gray56 '	'gray57 '
'gray58 '	'gray59 '	'gray60 '
'gray61 '	'gray62 '	'gray63 '
'gray64 '	'gray65 '	'gray66 '
'gray67 '	'gray68 '	'gray69 '
'gray70 '	'gray71 '	'gray72 '
'gray73 '	'gray74 '	'gray75 '
'gray76 '	'gray77 '	'gray78 '
'gray79 '	'gray80 '	'gray81 '

'gray82 '	'gray83 '	'gray84 '
'gray85 '	'gray86 '	'gray87 '
'gray88 '	'gray89 '	'gray90 '
'gray91 '	'gray92 '	'gray93 '
'gray94 '	'gray95 '	'gray97 '
'gray98 '	'gray99 '	

Appendix E More About the `import` Statement

A module is a Python source code file that contains functions and/or classes. Many of the functions in the Python standard library are stored in modules. For example, the `math` module contains various mathematical functions, and the `random` module contains functions for working with random numbers.

In order to use the functions and/or classes that are stored in a module, you must import the module. To import a module, you write an `import` statement at the top of your program. Here is an `import` statement that imports the `math` module:

```
import math
```

This statement causes the Python interpreter to load the contents of the `math` module into memory, making the functions and/or classes that are stored in the `math` module available to the program. To use any item that is in the module, you must use the item's *qualified name*. This means that you must prefix the item's name with the name of the module, followed by a dot. For example, the `math` module has a

function named `sqrt` that returns the square root of a number. To call the `sqrt` function, you would use the name `math.sqrt`. The following interactive session shows an example:

```
>>> import math
>>> x = math.sqrt(25)
>>> print(x)
5.0
>>>
```


Importing a Specific Function or Class

The previously shown form of the `import` statement loads the entire contents of a module into memory. Sometimes you want to import only a specific function or class from a module. When this is the case, you can use the `from` keyword with the `import` statement, as shown here:

```
from math import sqrt
```

This statement causes only the `sqrt` function to be imported from the `math` module. It also allows you to call the `sqrt` function without prefixing the function's name with the name of the module. The following interactive session shows an example:

```
>>> from math import sqrt
>>> x = sqrt(25)
>>> print(x)
5.0
>>>
```

You can specify the names of multiple items, separated with commas, when you use this form of the `import` statement. For example, the `import` statement in the following interactive session imports only the `sqrt` function and the `radians` function from the `math` module:

```
>>> from math import sqrt, radians
>>> x = sqrt(25)
>>> a = radians(180)
>>> print(x)
5.0
>>> print(a)
3.141592653589793
>>>
```

Wildcard Imports

A wildcard `import` statement loads the entire contents of a module. Here is an example:

```
from math import *
```

The difference between this statement and the `import math` statement is that the wildcard `import` statement does not require you to use the qualified names of the items in the module. For example, here is an interactive session that uses a wildcard `import` statement:

```
>>> from math import *  
>>> x = sqrt(25)  
>>> a = radians(180)  
>>>
```

And, here is an interactive session that uses the regular `import` statement:

```
>>> import math  
>>> x = math.sqrt(25)
```

```
>>> a = math.radians(180)
>>>
```

Typically, you should avoid using the wildcard `import` statement because it can lead to name clashes when you are importing several modules. A *name clash* occurs when a program imports two modules that have functions or classes with the same name. Name clashes do not occur when you are using the qualified names of a module's functions and/or classes.

Using an Alias

You can use the `as` keyword to assign an *alias* to a module when you import it. The following statement shows an example:

```
import math as mt
```

This statement loads the math module into memory, assigning the module the alias `mt`. To use any of the items that are in the module, you prefix the item's name with the alias, followed by a dot. For example, to call the `sqrt` function, you would use the name `mt.sqrt`. The following interactive session shows an example:

```
>>> import math as mt
>>> x = mt.sqrt(25)
>>> a = mt.radians(180)
>>> print(x)
5.0
>>> print(a)
3.141592653589793
>>>
```

You can also assign an alias to a specific function or class when you import it. The following statement imports the `sqrt` function from the `math` module, and assigns the function the alias `square_root`:

```
from math import sqrt as square_root
```

After using this `import` statement, you would use the name `square_root` to call the `sqrt` function. The following interactive session shows an example:

```
>>> from math import sqrt as square_root
>>> x = square_root(25)
>>> print(x)
5.0
>>>
```

In the following interactive session, we import two functions from the `math` module, giving each of them an alias. The `sqrt` function is imported as `square_root`, and the `tan` function is imported as `tangent`:

```
>>> from math import sqrt as square_root, tan as tangent
>>> x = square_root(25)
>>> y = tangent(45)
>>> print(x)
```

```
5.0
```

```
>>> print(y)
```

```
1.6197751905438615
```

Appendix F Installing Modules with the `pip` Utility

The Python standard library provides classes and functions that your programs can use to perform basic operations, as well as many advanced tasks. There are operations, however, that the standard library cannot perform. When you need to do something that is beyond the scope of the standard library, you have two choices: write the code yourself, or use code that someone else has already written.

Fortunately, there are thousands of Python modules, written by independent programmers, that provide capabilities that the standard Python library does not offer. These are known as *third-party modules*. A large collection of third-party modules exists at the website pypi.python.org, which is known as the *Python Package Index*, or *PyPI*.

The modules that are available at PyPI are organized as packages. A *package* is simply a collection of one or more related modules. The easiest way to download and install a package is with the `pip` utility.

The `pip` utility is a program that is part of the standard Python installation, beginning with Python 3.4. To install a package on a Windows system with the `pip` utility, you must open a command prompt window, and type a command following this format:


```
pip install package_name
```

`package_name` is the name of a package that you want to download and install. If you are running a Mac or a Linux system, you must use the `pip3` command instead of the `pip` command. In addition, you will need superuser privileges to execute the `pip3` command on a Mac or Linux system, so you will have to prefix the command with `sudo`, as shown here:

```
sudo pip3 install package_name
```

Once you enter the command, the `pip` utility will start downloading and installing the package. Depending on the size of the package, it may take a few minutes to complete the installation process. Once the process is finished, you can typically verify that the package was correctly installed by starting IDLE and entering the command

```
>>> import package_name
```

where `package_name` is the name of the package that you installed. If you do not see an error message, you can assume the package was successfully installed.

In **Chapter 7** , you will explore a popular third-party package named `matplotlib`. You can use the `matplotlib` package to create charts and graphs.

Appendix G Answers to Checkpoints